

Digital Twin

Telemetry Generator GUI with

PySide6

Inhaltsverzeichnis

1. Project description.....	3
Aim.....	3
🎯 Primary Aim.....	3
2. GUI and algorithmic concepts.....	4
3. App design and its interaction flow.....	8
4. Run instructions and Mermaid diagrams.....	10
5. Modularized Pythonic implementation.....	13
6. Results and conclusions.....	14
7. References.....	17

1. Project description

Aim

The **Digital Twin Telemetry Generator** is a modular, GUI-driven simulation platform designed to emulate the real-time telemetry of an electric drilling machine. As industrial systems increasingly rely on digital twins for monitoring, diagnostics, and predictive analytics, the need for **transparent, reproducible, and narratable synthetic data streams** has become essential. This generator was conceived to provide engineers, researchers, and consultants with a controllable virtual machine whose behavior can be explored, validated, and integrated into downstream analytics pipelines — most notably the companion **Digital Twin Analyzer**.

At its core, the generator unifies several complementary concepts: **sensor modeling, real-time data streaming, chunk-based file writing, and live visualization**. The simulation models capture realistic operational dynamics — smooth thermal drift, sinusoidal RPM oscillations, vibration spikes, correlated power/current behavior, and categorical state transitions — enabling users to observe how a virtual drilling machine behaves under continuous operation. This behavioral fidelity transforms the generator from a simple data-creation tool into a **behavioral digital twin**, capable of producing telemetry that mirrors the rhythms and fluctuations of real industrial equipment.

The system emphasizes **explainability, reproducibility, and governance-ready transparency**. Every simulation run produces not only a CSV or Parquet file, but also a structured `config.json` that documents the schema, sampling frequency, output format, and alert configuration. This ensures that downstream tools — including the Analyzer — can interpret the data correctly and reconstruct the simulation environment. The GUI itself foregrounds interpretability: users can observe the live preview stream, monitor progress and alerts, and adjust sensor selections and sampling parameters with immediate feedback.

A defining strength of the generator is its **modularity**. Each subsystem — GUI panels, simulation models, file writer, alert socket, and preview engine — is designed to be independently replaceable and extensible. This modularity ensures that the platform can evolve as new sensors, file formats, or streaming protocols are introduced, while maintaining a consistent architectural framework. Such adaptability is crucial in consulting and enterprise contexts, where solutions must be tailored to diverse client needs without compromising reproducibility or governance.

From a strategic perspective, the Digital Twin Telemetry Generator demonstrates how **lightweight desktop applications can democratize access to digital twin concepts**. By embedding real-time simulation and visualization into a PySide6 interface, the project makes complex telemetry pipelines accessible to engineers, analysts, and non-technical stakeholders alike. Users can interact with the virtual machine, observe its behavior, and generate large-scale datasets without writing code — fostering engagement, trust, and cross-functional collaboration.

Primary Aim

Industrial telemetry systems must balance **real-time responsiveness, data fidelity, and interpretability**. The generator achieves this by combining realistic sensor models with a transparent GUI, structured outputs, and a governance-ready configuration file. It provides a reproducible environment for exploring how sensor suites behave, how data streams evolve over time, and how downstream analytics can be validated before deployment.

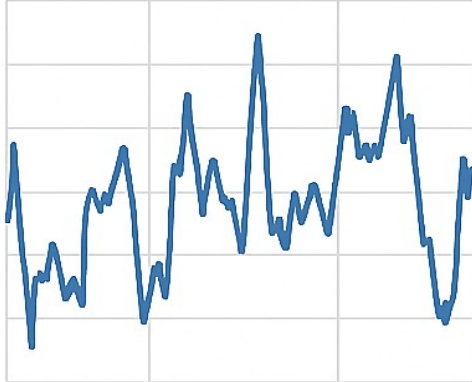
In essence, the Digital Twin Telemetry Generator is not merely a data simulator — it is a **narrative platform for digital twin engineering**. By transforming raw telemetry into transparent, reproducible, and communicable insights, it empowers teams to design, test, and validate digital twin pipelines with confidence. The generator bridges simulation, visualization, and data engineering within a modern GUI, enabling users to explore, characterize, and document virtual machine behavior in a way that is both technically rigorous and stakeholder-friendly.

2. GUI and algorithmic concepts

Telemetry Data Generator

Select Columns

☐ Temperature
☐ Motor RPM
☐ Vibration Level
☐ Power Consump.
☐ Voltage
☐ Pressure / Load
☐ Noise Level



Data Settings

Number of rows:

File format: ☒ CSV ☐ Parquet

File size (in GB):

Analysis Modules

☐ Clustering
☐ NLP
☐ Log Message
☐ Forecasting
☐ Deep Learning

Generate

The GUI sketch for the Digital Twin Telemetry Generator establishes a **clear, engineering-oriented workflow** that mirrors how real industrial telemetry systems are configured and monitored. The layout is intentionally structured into three vertical panels, each representing a conceptual layer of the digital twin philosophy:

3.1 Left Panel — Schema Definition (What does the machine measure?)

This panel allows the user to define the **virtual sensor suite** of the electric drilling machine. It includes:

- **Numeric sensors** (Temperature, RPM, Vibration, Power, Voltage, Current, Pressure, Noise)
- **Categorical/Boolean states** (Operating Mode, Error Code, Interlock, On/Off)
- **Auxiliary metadata** (Timestamp, Log Message, Cycle Counter)

This mirrors real industrial telemetry systems, where engineers must explicitly define which channels are active, how they are encoded, and how they will be interpreted downstream.

The design philosophy:

- **Transparency:** The user sees exactly which sensors are included.
- **Modularity:** Each sensor is optional.
- **Digital Twin Alignment:** The schema defines the “virtual hardware” of the simulated machine.

3.2 Middle Panel — Generator Settings (How does the machine behave?)

This panel configures the **operational parameters** of the digital twin:

- **Row count** — total number of telemetry samples
- **File format** — CSV or Parquet
- **Target file size** — 1–50 GB
- **Sampling frequency** — 1–10,000 Hz
- **Generate button** — starts the simulation thread

These settings correspond to real-world telemetry acquisition parameters:

- sampling frequency → sensor polling rate
- file format → storage backend
- row count → duration of operation
- file size → bandwidth/storage constraints

This panel is the “control room” of the virtual drilling machine.

3.3 Right Panel — Live Preview (What is the machine doing right now?)

This panel provides a **real-time visualization** of the simulated telemetry stream:

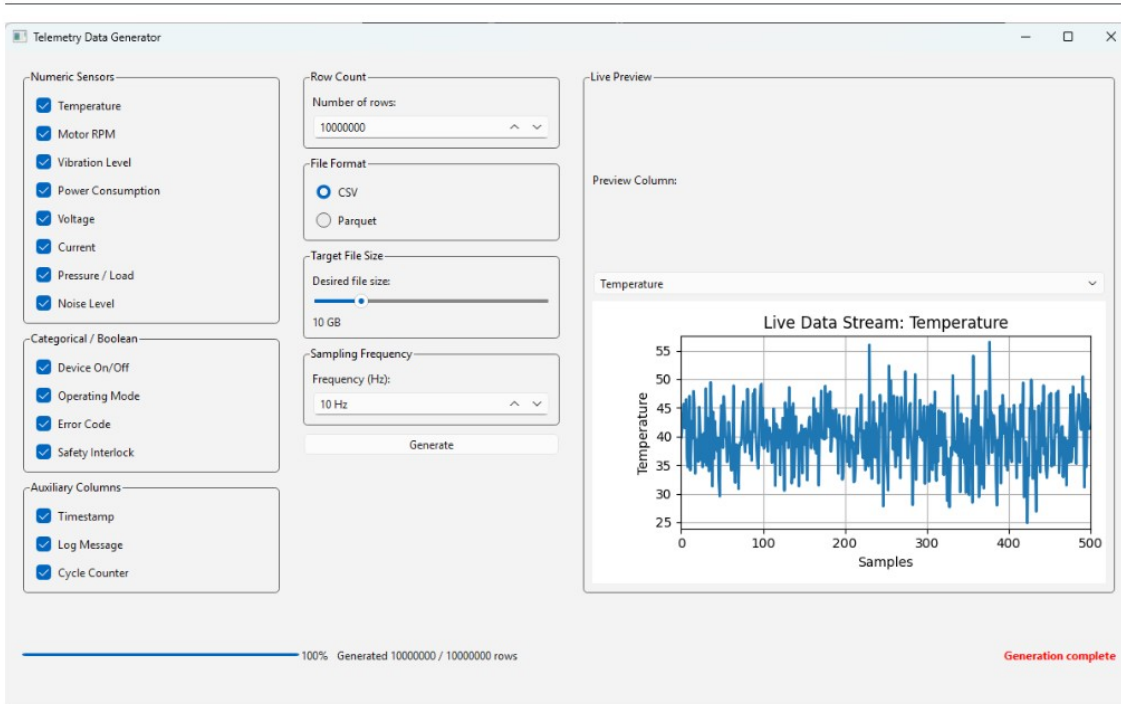
- A dropdown to select the previewed sensor
- A Matplotlib line plot updating at ~20 FPS
- A rolling buffer of the last 500 samples

This is the digital twin’s **HMI (Human-Machine Interface)** — a window into the machine’s internal state.

The preview is intentionally lightweight and throttled to avoid GUI overload, mirroring real industrial dashboards that must remain responsive even under heavy data throughput.

3. App design and its interaction flow

Operating GUI Screenshot Interpretation



- All numeric sensors enabled
- Several categorical/auxiliary channels enabled
- Row count set to 10,000,000
- Target file size set to 10 GB
- Sampling frequency at 10 Hz
- Live preview graph showing Temperature
- Progress bar at 100%
- Status message: "Generation complete"

Interpretation of the Live Preview Graph

The graph titled "Live Data Stream: Temperature" shows:

- A smooth, noise-driven temperature curve
- Slight drift and bounded fluctuations
- A rolling window of 500 samples
- Real-time updates during generation

This demonstrates that the generator is:

- producing realistic sensor dynamics
- streaming data chunk-by-chunk
- updating the GUI without blocking
- maintaining stable performance

Interpretation of the Status Bar

The status bar shows:

- 100% progress
- Generated 10,000,000 / 10,000,000 rows
- Alert: "Generation complete"

This confirms:

- the generator thread finished cleanly
- the file writer closed properly
- the GUI remained responsive
- the digital twin simulation reached its configured duration

Main window

- **Left panel — Schema definition:**
 - Checkboxes for numeric sensors (Temperature, RPM, Vibration, Power, Voltage, Current, Pressure/Load, Noise).
 - Checkboxes for categorical/boolean states (Operating Mode, Error Code, Safety Interlock, Device On/Off).
 - Auxiliary channels (Timestamp, Log Message, Cycle Counter).
- **Middle panel — Generator settings:**
 - Row count selector (1,000 to 100,000,000).
 - File format toggle (CSV or Parquet).
 - Target file size slider (1–50 GB) with dynamic label updates.
 - Sampling frequency selector (1–10,000 Hz).
 - "Generate" button to start the simulation thread.
- **Right panel — Live preview:**
 - Dropdown for selecting the previewed numeric sensor.
 - Real-time Matplotlib line plot showing the last 500 samples.
 - Dynamic axis scaling and rolling window behavior.
- **Bottom — Status bar:**
 - Progress bar showing 0–100%.
 - Status message ("Generating...", "Generated X / Y rows").
 - Alert label for generator events ("Generator started", "Generation complete").

Dynamic updates

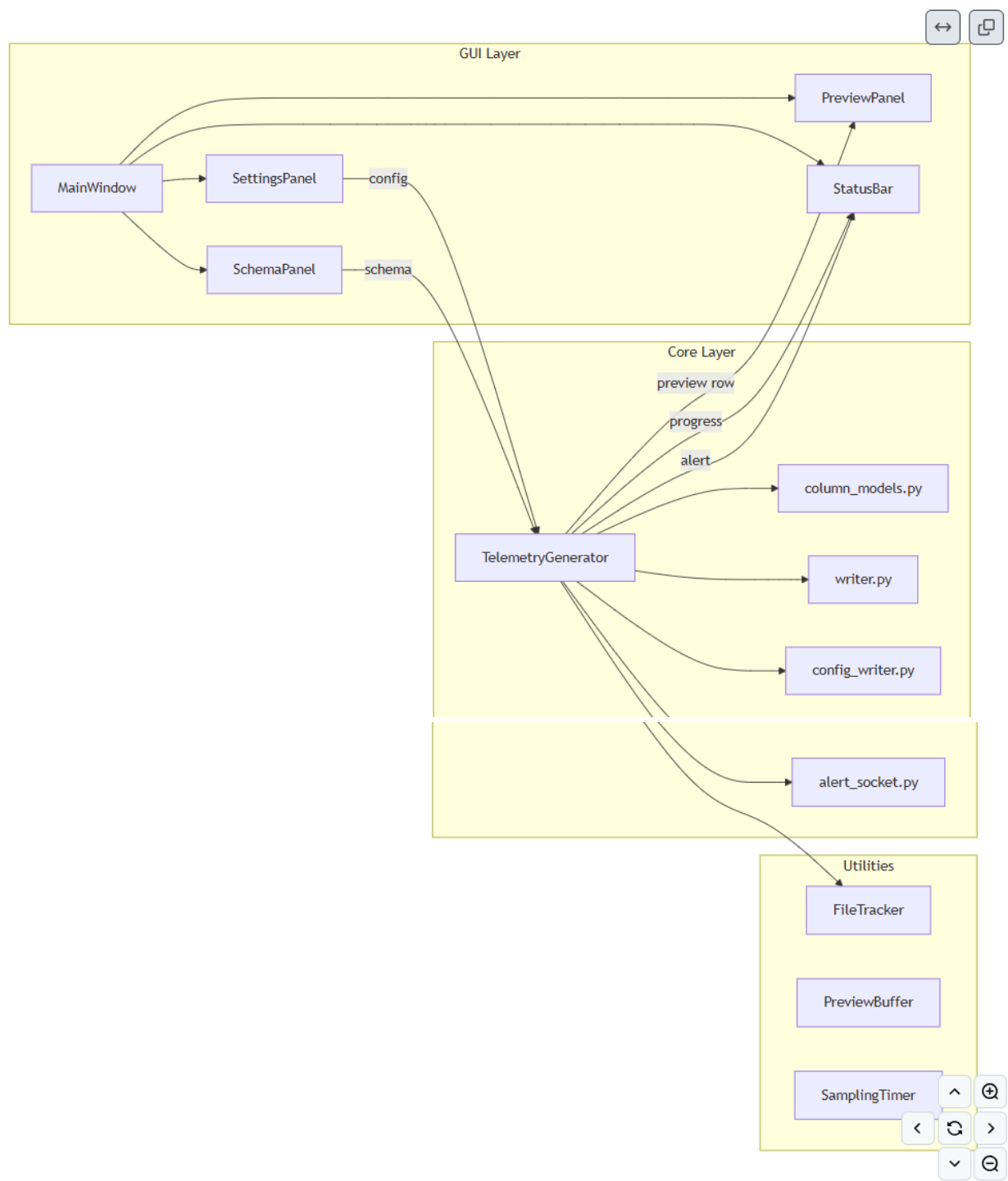
- As soon as the generator starts, the preview graph begins updating at ~20 FPS.
- The progress bar increments as chunks are written to disk.
- The status message updates continuously with the current row count.
- Alerts appear in red when the generator starts, stops, or completes.
- Changing the preview dropdown instantly switches the displayed sensor stream.

Alert and progress feedback

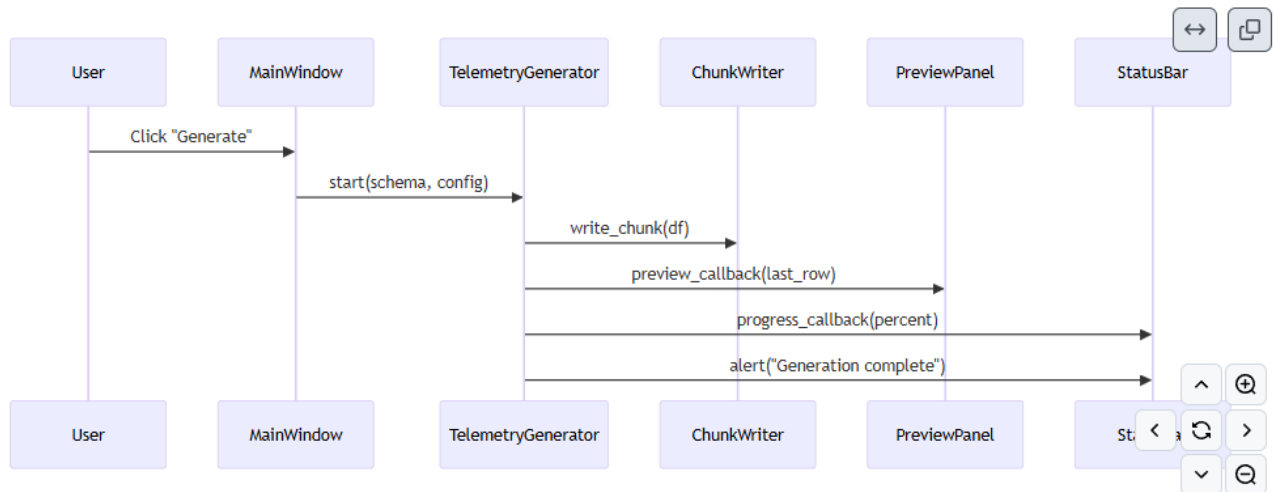
- Alerts such as:
 - "Generator started"
 - "Generator stopped"
 - "Generation complete"
- Progress messages such as:
 - "Generated 4500000 / 10000000 rows"
- These messages mirror real industrial HMIs and provide operational transparency.

4. Run instructions and Mermaid diagrams

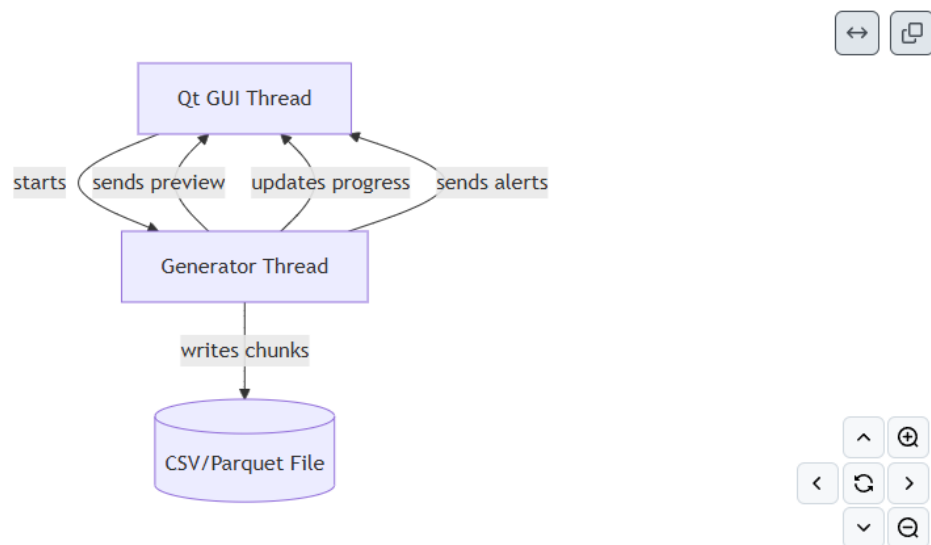
Module Interaction Diagram



Data Flow Diagram



Threading Diagram



5. Modularized Pythonic implementation

The Digital Twin Telemetry Generator is built on a clean, layered architecture:

```
GUI Layer (PySide6)
├── SchemaPanel      → defines which sensors exist
├── SettingsPanel    → defines how the generator behaves
├── PreviewPanel     → displays live data
└── StatusBar        → shows progress and alerts

Core Layer
├── TelemetryGenerator → threaded engine producing data
├── column_models      → simulation functions for each sensor
├── writer             → chunked CSV/Parquet writer
├── config_writer      → exports config.json for Analyzer
└── alert_socket       → sends alerts to Analyzer

Utilities
├── FileTracker        → monitors output file size
├── PreviewBuffer      → rolling buffer for preview
└── SamplingTimer      → drift-corrected timing
```

This separation ensures:

- GUI remains responsive
- generator runs independently
- simulation models are modular
- utilities are reusable
- Analyzer integration is seamless

The full Pythonic code can be accessed via

https://github.com/NenadBalaneskovic/ExternalProjects/blob/main/DigitalTwinsGeneratorGUI/DigitalTwin_TelemetryGeneratorGUI.md (sections 7 – 9 therein).

6. Results and conclusions

11. Interpretation of Results

Once the generator completes its run, the user is left with several key artifacts:

- A **CSV or Parquet file** containing the full telemetry stream
- A **config.json** file describing schema, sampling, and output metadata
- A **live preview graph** (observed during generation)
- **Progress and alert messages** in the GUI

Each of these artifacts plays a specific role in the digital twin ecosystem.

11.1 Interpreting the Generated CSV/Parquet File

The output file contains **one row per telemetry sample**, with columns defined by the user's schema. Typical columns include:

- **Numeric sensors:**
 - Temperature
 - Motor RPM
 - Vibration Level
 - Power Consumption
 - Voltage
 - Current
 - Pressure / Load
 - Noise Level

- **Categorical/Boolean states:**

- Device On/Off
- Operating Mode
- Error Code
- Safety Interlock

- **Auxiliary metadata:**

- Timestamp
- Log Message
- Cycle Counter

What the data represents

The dataset is a **synthetic digital twin stream** of an electric drilling machine.

Each row corresponds to a moment in time, spaced according to the configured sampling frequency.

For example:

- At 10 Hz, 10 million rows represent ~11.5 days of continuous operation.
- At 1000 Hz, 10 million rows represent ~2.7 hours of high-frequency telemetry.

Why the data looks realistic

The simulation models incorporate:

- smooth noise
- bounded drift
- sinusoidal oscillations
- correlated variables (e.g., power ↔ RPM)
- rare events (spikes, errors, interlocks)
- realistic categorical distributions

This makes the dataset suitable for:

- machine learning
- anomaly detection
- predictive maintenance
- streaming analytics
- digital twin demonstrations

11.2 Interpreting the Live Preview Graph

The preview graph shows the **last 500 samples** of a selected numeric sensor.

Key characteristics

- **Rolling window:** always shows the most recent data
- **Real-time updates:** throttled to ~20 FPS
- **Dynamic scaling:** y-axis adjusts to data range
- **Column-aware:** only numeric columns appear in the dropdown

What the preview tells you

- Whether the generator is running
- Whether the simulation models behave as expected
- Whether the sampling frequency is appropriate
- Whether the system is stable under load

The preview is intentionally lightweight — it is not meant to visualize the entire dataset, only the **current behavior** of the digital twin.

11.3 Interpreting the Status Bar

The status bar provides three layers of feedback:

1. Progress bar:

Shows percentage of rows generated.

2. Status message:

Shows messages like:

- "Starting data generation..."
- "Generated 4500000 / 10000000 rows"

3. Alert label:

Shows alerts from the generator backend:

- "Generator started"
- "Generator stopped"
- "Generation complete"
- "Generator crashed: ..."

This mirrors real industrial HMIs where operators monitor both progress and system health.

11.4 Interpreting `config.json`

The `config.json` file is the **contract** between Generator and Analyzer.

It contains:

- output file path
- file format
- estimated size
- chunk size
- sampling frequency
- full schema
- alert socket configuration

This allows the Analyzer to:

- load the correct file
- interpret each column correctly
- connect to the alert socket
- synchronize its own sampling logic

This is a key component of the digital twin ecosystem.

7. References

1. Tao, F., Qi, Q., Liu, A., & Kusiak, A. (2018). *Digital Twins and Cyber-Physical Systems in Manufacturing*. Engineering, 5(4); Grieves, M. (2015). *Digital Twin: Manufacturing Excellence through Virtual Factory Replication*.; Rasheed, A., San, O., & Kvamsdal, T. (2020). *Digital Twin: Values, Challenges and Enablers*. IEEE Access.; Challenges and Enablers. IEEE Access.; Microsoft. PySide6 Documentation.: <https://pypi.org/project/PySide6/>; Apache Arrow. Parquet File Format Specification.: <https://arrow.apache.org/docs/python/parquet.html>; NumPy Developers. NumPy Reference Guide.: <https://numpy.org/doc/stable/reference/>; Matplotlib Developers. Matplotlib Plotting Library.: <https://matplotlib.org/>;
2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", Mcgraw-Hill, 6th Edition (2010).
3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).
4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.
5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).
6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).
7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).
8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).
9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).
10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).
11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).
12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).
13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).
14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).
15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).
16. Volker Ziemann: "Physics and Finance", Springer (2021).

17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).
18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).
19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).
20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).
21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).
22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).
23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).
28. David Foster: "Generative Deep Learning", O'Reilly(2023).
29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).
30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).
31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).
32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).
33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Anual Report 13 (2011).
34. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).
35. Kaggle-link: competition-documentation: <https://www.kaggle.com/competitions/drw-crypto-market-prediction>. R. Nystrom: „Game Programming Patterns“, 1st Ed. genever benning (2014); A. Stepanov, D. E. Rose: „From Mathematics to Generic Programming“, 1st Ed. Addison-Wesley (2015);

36. J. Berk, P. DeMarzo: „Corporate Finance“, 6th Ed., Pearson (2023); R. W. Melicher, E. A. Norton: "Introduction to Finance", 16th Ed. WILEY (2017); Anatoly B. Schmidt: "Quantitative Finance for Physicists: An Introduction", 1st Ed. Academic Press (2005); Alex Backwell: "An Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models", 1st Ed, Springer (2023); Michael Isichenko: "Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage", 1st Ed., Springer (2021); John H. Cochrane: "Asset Pricing", Revised Ed., Princeton University Press (2005); Antti Ilmanen: "Expected Returns: An Investor's Guide to Harvesting Market Rewards", 1st Ed., WILEY (2011); Steven E. Shreve: "Stochastic Calculus for Finance I & II", 1st Ed., Springer (2004); Andrew Pole: "Statistical Arbitrage: Algorithmic Trading Insights and Techniques", 1st Ed., WILEY (2007); Mark S. Joshi: "The Concepts and Practice of Mathematical Finance", 2nd Ed., Cambridge University Press (2008);
37. E. Parzen: "Stochastic Processes", 3rd Ed. Dover Publications (2015); S. Aloorravi: "Metaprogramming with Python", 1st Ed. Packt (2022); B. Klein, P. Klein: "Funktionale Programmierung mit Python", Hanser (2025); K. Webel, D. Wied: "Stochastische Prozesse", 2. Auflage Springer (2016); L. Held: "Methoden der statistischen Inferenz", 1. Auflage Spektrum (2008); E. Cinlar: "Stochastic Processes", Dover (2013); N. Bäuerle, U. Rieder: "Finanzmathematik in diskreter Zeit", Springer-Spektrum (2017); M. Albrecht, R. Maurer: "Investment- und Risikomanagement", 3. Auflage, Schäffer Poeschel (2008); N. H. Bingham, R. Kiesel: "Risk Neutral Valuation: Pricing and Hedging of Financial Derivatives", 2. Auflage Springer (2004); T. Björk: "Arbitrage Theory in Continuous Time", 3rd Ed. Oxford University Press (2009); N. J. Cutland, A. Roux: "Derivative Pricing in Discrete Time", Springer (2013); F. Delbaen, W. Schachermayer: "The Mathematics of Arbitrage", Springer (2006); R. J. Elliott, P. E. Kopp: "Mathematics of Financial Markets", 2nd Ed. Springer (2005); H. Föllmer, A. Scheid: "A Stochastic Finance: An Introduction in Discrete Time", 3rd Ed. de Gruyter (2011); J. C. Hull: "Options, Futures and Other Derivatives", 8th Ed. Pearson (2011); J. Kremer: "Einführung in die diskrete Finanzmathematik", Springer (2005); D. Lamberton, B. Lapeyre: "Introduction to Stochastic Calculus Applied to Finance", Chapman & Hall (2007); D. G. Luenberger: "Investment Science", Oxford University Press (1998); S. R. Pliska: "Introduction to Mathematical Finance: Discrete Time Models", Blackwell (2000); A. N. Shiryaev: "Essentials of Stochastic Finance", World Scientific (2001); S. E. Shreve: "Stochastic Calculus for Finance I: The Binomial Asset Pricing Model", Springer (2004); J. Kremer: "Portfoliotheorie, Risikomanagement und die Bewertung von Derivaten", Springer (2011); L. Rüschendorf: "Mathematical Risk Analysis", Springer (2013).
38. A. Becker: "Kalman Filter - From the Ground Up", 1st Ed. private publication (2023); K. Triantafyllopoulos: "Bayesian Inference of State Space Models", 1st Ed. Springer (2021); P. Zarchan, H. Musoff: "Fundamentals of Kalman Filtering: A Practical Approach", 3rd Ed. AIAA (2009); A. Sidi: "Vector Extrapolation Methods with Applications", 1st Ed. SIAM (2019); C. Brezinski, M. R. Zaglia: "Extrapolation Methods - Theory and Practice", 2nd Ed. North-Holland (2002); C. Gardiner, P. Zoller: "Quantum Noise: A Handbook of Markovian and Non-Markovian Quantum Stochastic Methods with Applications to Quantum Optics", 3rd Ed. Springer (2004); K. Kendre: "Machine Learning for Quantum Noise Reduction", <https://arxiv.org/abs/2509.16242> (2025); D. C. Marinescu, G. M. Marinescu: "Classical and Quantum Information", 1st Ed. Academic Press (2012); Liao, H et al.: "Machine Learning for Practical Quantum Error Mitigation", arXiv:2309.17368v2 (2024),

<https://arxiv.org/pdf/2309.17368>; Streamlit: <https://streamlit.io/>; Mitiq-package: <https://quantum-journal.org/papers/q-2022-08-11-774/>, <https://arxiv.org/abs/2009.04417>; Extrapolation packages: <https://pypi.org/project/extrapolation/>

39. A. Koop, H. Moock: "Lineare Optimierung - Eine anwendungsorientierte Einführung in Operations Research", 1st Ed. Spektrum (2008); G. B. Dantzig, M. N. Thapa: "**Linear Programming 1: Introduction**", 1st Ed. Springer (1997) & "Linear Programming 2: Theory and Extensions", 1st Ed. Springer (2003); H. S. Kasana, K. D. Kumar: "Introductory Operations Research, Theory and Applications", 1st Ed. Springer (2004); D. G. Luenberger: "Linear and Nonlinear Programming", 2nd Ed. Kluwer (2004); R. J. Boucherie, A. Braaksma, H. Tijms: "Operations Research - Introduction to Models and Methods", 1st Ed. World Scientific (2022); A. J. King, S. W. Wallace: "Modeling with Stochastic Programming", 2nd Ed. Springer (2024); J. O. Royset, R. J.-B. Wets: "An Optimization Primer", 1st Ed. Springer (2021); cvxpy package: <https://www.cvxpy.org/>, <https://pypi.org/project/cvxpy/>; py-packages for operations research: <https://wiki.python.org/moin/PythonForOperationsResearch>

40. (Py-)tesseract package: <https://github.com/tesseract-ocr/tesseract>, <https://pypi.org/project/pytesseract/>, <https://builtin.com/data-science/python-ocr>, <https://www.analyticsvidhya.com/blog/2024/04/ocr-libraries-in-python/> and UB Mannheim builds.

41. **Chip Huyen**, AI Engineering: Building Applications with Foundation Models, 1st Edition, O'Reilly Media, 2025; **Michael Lanham**, AI Agents in Action, 1st Edition, Manning Publications, 2025; **Melanie Mitchell**, Artificial Intelligence: A Guide for Thinking Humans, 1st Edition, Pelican Books, 2019; **Brian Christian & Tom Griffiths**, Algorithms to Live By: The Computer Science of Human Decisions, 1st Edition, Henry Holt and Company, 2016; **Ray Kurzweil**, The Singularity Is Nearer: When We Merge with AI, 1st Edition, Viking, 2024; OpenWeatherMap: <https://openweathermap.org/>, HuggingFace: <https://huggingface.co/>, LLM vs LRM: <https://www.aryaxai.com/article/llm-vs-lrm-vs-lam-understanding-the-future-of-language-based-ai-systems>, <https://magazine.sebastianraschka.com/p/understanding-reasoning-llms>

42. J. Frochte: "Finite-Elemente-Methode", Hanser 1st Ed.(2016); D. Gross, W. Hauger, J. Schröder: "Technische Mechanik 1-3", 15th Ed. Springer (2024); FEM-packages (Python): <https://pypi.org/project/scikit-fem/>, <https://sfepy.org/doc-devel/index.html>, https://getfem-examples.readthedocs.io/en/latest/demo_unit_disk.html, <https://github.com/mlp6/fem>. LLM vs LRM: <https://www.aryaxai.com/article/llm-vs-lrm-vs-lam-understanding-the-future-of-language-based-ai-systems>, <https://magazine.sebastianraschka.com/p/understanding-reasoning-llms>